

007

Harness + SSD

全栈应用实战

按规格完成一个有前端、后端和数据的应用

本课要完成的课程平台

访客

浏览课程列表
查看课程详情
注册或登录

学员

提交课程报名
查看自己的报名
识别失败并重试

管理员

编辑与发布课程
查看报名记录
限制后台操作权限

页面、业务逻辑和数据库，共同完成一次报名

Harness 与 SSD 的分工

层次	在全栈开发中解决什么
SSD 规格驱动	写清用户行为、数据、接口和验收条件
Harness 执行约束	规定读取顺序、改动范围、状态和质量门
全栈应用实现	让页面、服务端和数据库按规格协同工作
交付证据	说明哪个版本完成了哪些用户场景

全栈应用的开发路线

01

产品定义

P0-P3 内容与蓝图

02

SSD 规格

P4 架构与施工计划

03

全栈实现

P5-P6 贯通业务

04

验证上线

P7-P8 交付与迭代

每个阶段，都有明确输入、产物和出口条件

01

产品定义与页面蓝图

先明确用户要完成什么，再确定页面和技术范围

第一版范围

本期实现	以后扩展
课程列表与详情	推荐、搜索和复杂分类
登录后报名	支付、退款与优惠券
我的报名记录	学习进度与证书
管理员发布课程	复杂审批与运营分析

先让一条业务链真正可用，再扩大功能范围

一次报名的用户路径

01

浏览课程

看到可报名的课程

02

确认身份

登录后提交报名

03

服务端保存

校验后写入数据库

04

查看结果

刷新后仍能查到

刷新后仍然成立，才证明数据已经持久化

P0：产品事实与内容清单

用户 访客、学员、管理员分别要完成什么

内容 课程标题、介绍、时间和报名规则

事实 哪些已确认，哪些仍需要负责人决定

边界 免费报名，无名额上限，暂不接支付

这些决定会进入需求、页面文案和数据模型

P1-P2：品牌与设计约束

品牌方向

面向学习者的表达、品牌资产与边界

设计 token

颜色、字号、间距采用命名规则

共享组件

导航、按钮、表单和状态反馈统一



案例原有品牌资产

页面的差异来自内容，共同规则来自统一来源

P3：关键页面与交互状态

公开页面

课程列表
课程详情
登录入口

学员页面

未报名与已报名
提交中与提交失败
我的报名空状态

管理页面

草稿与已发布
编辑校验失败
无权限访问

蓝图审核同时覆盖页面效果和用户状态

页面路由与权限范围

页面	访问者	数据来源
/courses	所有人	已发布课程
/courses/[id]	所有人	单门已发布课程
/auth/login	未登录用户	账户与会话
/dashboard/enrollments	当前学员	自己的报名记录
/admin/courses	管理员	课程草稿与发布状态

02

SSD：把应用写成规格

需求、数据、接口与任务，应当互相对应

一项功能的 SSD 规格包

文件	课程报名功能的内容
00-discovery.md	现有账户、页面、数据与依赖
01-requirements.md	报名行为、权限和验收标准
02-design.md	数据模型、业务接口与错误处理
03-tasks.md	数据库、后端、页面与测试任务
04-fpf-review.md	反查规格矛盾和遗漏的失败场景

给 Agent 的规格任务

目标：增加免费课程报名功能。
先读取项目规则、账户与数据模块。
使用 spec-dev 形成需求与设计。
写清数据、权限、接口和失败场景。
拆分任务，列出文件与验收条件。
把未决问题和假设单独列出。

提供输入

产品范围、蓝图与当前代码

审核产物

规格是否覆盖完整用户流程

把需求与边界讲清楚，Agent 才能形成可实施的计划

一条可测试的报名需求

WHEN 已登录学员提交课程报名
THE 系统 SHALL 校验课程可报名
AND 保存当前用户的报名记录

IF 已存在报名记录
THEN 返回已有结果，不重复新增

成功路径

报名后刷新，仍可查到

失败路径

未登录、课程下架、写入失败

一句“做个报名功能”，需要展开成条件与结果

数据模型与约束

实体	核心字段	重要约束
user	id、账户与角色	复用现有身份体系
course	id、标题、状态	草稿与发布状态明确
enrollment	userId、courseId、时间	用户与课程组合唯一
关联	报名关联用户与课程	外键与删除策略明确

业务规则应落到服务端和数据库约束中

权限矩阵

操作	访客	学员	管理员
查看已发布课程	允许	允许	允许
提交报名	需登录	以本人身份	按已定义角色规则
查看报名记录	拒绝	仅自己的记录	按管理职责授权
编辑和发布课程	拒绝	拒绝	允许

报名接口的输入与输出

输入 : courselid
身份 : 服务端 session.user.id
成功 : enrollmentId + 已报名状态
重复 : 返回原有报名结果
失败 : 未登录 / 不可报名 / 保存失败

前端负责

提交参数与展示结果
失败后支持重试

服务端负责

校验身份与业务
执行真实写入

接口契约明确后，前后端才能独立实现

全栈架构的三个实现位置

页面与组件

src/app/
src/components/
用户输入与状态反馈

服务端业务

src/actions/
src/lib/
会话、校验与业务规则

数据与集成

src/db/
src/mail/、payment/
持久化与外部服务

先贯通业务调用，再决定是否要拆分更多层

任务按依赖顺序拆分

- T-01 数据** 定义 course、enrollment 与迁移
- T-02 业务** 实现身份、课程状态和重复报名处理
- T-03 页面** 接入报名动作和报名记录查询
- T-04 验证** 覆盖成功、失败、权限和重复请求

每张任务卡都标明文件、验收标准和依赖

四套 Skill 在开发中的使用时机

遇到的任务	使用的 Skill	产物
接手底座，盘点差距	blindspot-audit	现状与缺口证据
把缺口转成开发计划	requirements-matrix-generator	需求矩阵与依赖
设计并实施一项功能	spec-dev	规格、任务、验证
连续完成多项已定义任务	loop-prompt-generator	执行契约与恢复规则

03

贯通前端、后端与数据库

先完成一条报名链，再扩展页面和功能

底座的目录地图

src/app/	页面与路由
src/actions/	服务端动作
src/lib/	会话与通用能力
src/db/	数据模型与迁移
src/components/	共享界面组件
messages/	国际化词条

先确认可复用部分

账户、会话、组件、数据连接

再登记新增部分

课程、报名和管理页面

模板提供通用能力，业务仍需按自己的规格实现

本地开发的启动基线

```
cd template
cp env.example .env.local
pnpm install --frozen-lockfile
# 填写开发环境配置并准备数据库
pnpm dev
pnpm gate:check
```

先验证底座

记录启动结果和已有问题

再开始业务改动

保存基线，区分新旧问题

页面打开、账户可用、数据库可连，基线才清楚

环境配置与数据库迁移

开发环境	使用独立数据库和测试服务配置
迁移文件	新增实体后生成迁移并审查 SQL
应用迁移	确认目标环境，再执行相应迁移
恢复准备	记录版本、影响和数据恢复方式

数据库结构变化需要独立审查，不能只看页面效果

第一个贯通的业务功能

01

页面输入

提交课程 ID

02

服务端校验

确认会话与课程状态

03

数据库写入

唯一约束防止重复

04

重新查询

显示真实报名记录

同一条验收标准贯穿四个位置

数据库实现的检查点

设计决定	实现位置	验收方式
课程有发布状态	course 模型	草稿不可报名
用户与课程组合唯一	enrollment 唯一约束	并发重复只保留一条
关系有效	外键与查询条件	无效课程无法写入
迁移可应用	迁移 SQL 与版本记录	开发库迁移后可读写

服务端报名逻辑

```
user = requireSession()  
input = validate(courseId)  
course = requirePublishedCourse(input)  
result = enrollOnce(user.id, course.id)  
return enrollmentResult(result)
```

身份可信

userId 来自服务端会话

结果可信

保存失败应明确返回失败

参数校验、身份校验和业务校验，分别负责不同问题

前端接入的状态设计

可提交

显示课程与报名条件
按钮状态明确
登录后继续操作

提交中

防止重复触发
保留用户操作上下文
等待真实返回结果

结果反馈

成功后显示报名记录
失败给出可操作提示
允许重试或重新登录

服务端防重复仍然必要，禁用按钮只是交互措施

“我的报名”页面的数据边界

查询身份	服务端从当前会话取得用户
过滤范围	查询条件包含当前用户 ID
返回内容	只返回页面需要的字段
验证方法	换一个账户，不能读到别人的记录

页面隐藏和路由保护之外，还要检查数据访问范围

管理员发布课程

入口	后台页面检查登录与管理角色
动作	服务端再次检查发布权限
规则	发布前校验必填内容和有效状态
结果	公开页可见，草稿仍保持不可报名

发布是一项业务操作，需要自己的规格与回归用例

邮件与支付的扩展接入

能力	接入前补充的规格	关键验证
报名邮件	发送失败、重试与重复发送规则	业务成功与送达分别记录
付费报名	订单、支付回调、权益与退款	可信回调与幂等处理
文件存储	资源权限、大小和访问范围	上传与下载权限一致

外部服务接通后，还要验证完整业务结果

P6：多语言与 SEO 同步

应用部分	实现方式	验证重点
公开课程页面	词条、路由与页面元数据对应	语言内容与链接正确
私有报名与后台	权限控制，按规则设置索引策略	匿名访问不能读取私有数据
全站配置	统一维护导航、域名与页面信息	元数据与实际页面一致

Harness 如何约束这一轮开发

开工前

读取当前规格与基线
确认文件和接口边界
确认依赖已经满足

执行中

按任务更新代码
发现差异记录到规格
多人共享变更先协调

结束时

验证对应用户场景
记录证据和代码版本
回填任务状态与索引

中断后从任务文件和当前代码恢复

给 Agent 的实施任务

读取当前规格、任务卡与代码版本。
完成报名功能对应的下一项任务。
只修改任务涉及的文件和必要依赖。
按验收标准验证成功与失败路径。
记录命令、结果、证据和遗留项。
回写任务状态；阻塞时说明原因。

指定范围

用任务 ID 关联规格和文件

审阅结果

检查代码、行为与验证证据

任务状态、实际代码与验证结果需要一致

04

验证、部署与持续迭代

交付的是目标版本中能够工作的用户流程

报名功能的验收矩阵

场景	预期结果	观察位置
正常报名后刷新	记录仍在	页面与数据库
未登录直接请求	拒绝写入	服务端与数据库
重复或并发报名	只有一条有效记录	数据库与返回结果
存储写入失败	明确失败，可重试	接口与界面
课程变成草稿	按规格拒绝新报名	业务逻辑与数据

命令须与当前 package.json 对齐

本地质量门与业务测试

```
pnpm gate:check  
pnpm exec tsc --noEmit  
pnpm build
```

```
# 再运行项目新增的报名业务测试  
# 记录命令、结果与 commit SHA
```

静态门

代码和配置能否通过检查

业务门

报名和权限行为是否符合规格

检查项目实际提供的命令，再写入执行契约

门禁的覆盖范围要说清楚

现有检查

核心配置文件存在
部分代码模式检查
依赖数量与语言文件存在

仍需补充

业务成功和失败场景
用户数据与管理权限
语言完整性与集成回归

给检查器输入已知错误，确认它会阻止通过

P7：目标环境的发布流程

01

集成版本

合并并验证目标提交

02

环境配置

核对数据库与服务

03

执行发布

等待目标部署可用

04

线上验收

复查业务并记录版本

发布记录关联部署地址、commit SHA 和回滚方案

多分支成果需要共同验收

各自发布的后果

A 分支有 SEO，缺双语

B 分支有双语，缺 SEO

先后上线造成互相覆盖

全栈交付的做法

把需要的变更集成到同一版本

对这个版本运行完整回归

上线后复核两项功能

课程页、报名接口和数据库，也要按同一版本交付

P8 : 让下一轮开发更可靠

观测 记录报名失败、接口错误和关键业务状态

复盘 把触发条件、根因与修复方法写清楚

回写 更新对应规格、项目规则和导航索引

回归 把旧问题变成下一轮能发现的测试

新需求继续进入规格、实施和验证的循环

课堂练习：课程报名功能

前 10 分钟 写出需求、数据约束与接口契约

中 15 分钟 完成最小实现，或形成逐文件施工计划

后 5 分钟 复查成功、失败、权限和重复请求

未运行的代码和测试，明确标为未验证

一次全栈交付留下什么

交付物	他人如何复查
应用规格与任务	知道做什么、范围和遗留项
代码与数据迁移	能够定位实现和版本
验证证据	能够复现关键用户场景
运行与发布说明	能够启动、配置、部署和恢复

另一位协作者能够接手，是交付的一部分

007 资料怎样配合这条主线

项目流程 pipeline-spec/ 与 design-system/

规格与实施 skills/ 与 harness-engineering/

应用底座 template/ , 以自身许可为准

发布与复盘 operations/ 与 article/

从一个业务功能开始 , 把整条开发流程跑通

本期回顾

按规格完成 一个全栈应用

用户流程能跑通，数据与权限有验证，版本可交接

Harness 贯穿开发过程，SSD 把需求变成可实施规格